

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data MiningQ1

K-Means vs Hierarchical Clustering:

K-Means is faster and more scalable for large datasets because it directly assigns data points to a predefined number of clusters. Hierarchical clustering builds a hierarchy of clusters and is more computationally expensive, making it less suitable for very large datasets. For this small customer dataset, both methods produce similar 2-cluster results, so either can be used; however, K-Means is preferable when scalability and speed are important.

Method	Performance	Scalability
K-Means	Fast	High
Hierarchical	Slower	Lower

The screenshot displays the Weka Explorer interface with the 'Cluster' tab selected. The 'Clusterer' dropdown is set to 'HierarchicalClusterer'. The 'Cluster mode' section shows 'Use training set' selected. The 'Result list' on the left shows two results: '183118 - SimpleKMeans' and '183122 - HierarchicalClusterer'. The 'HierarchicalClusterer' result is selected, and its output is displayed in the main pane. The output shows the clustering model (full training set) and the final cluster centroids. The centroids are: Cluster 0: 0.65, Cluster 1: 0.864865, 40. The output also shows the number of iterations (2), the within cluster sum of squared errors (0.27626737675347706), and the initial starting points (random). The final cluster centroids are: Cluster# 0 1, Attribute Full Data (5.0) (3.0) (2.0), Income 0.5027 0.2162 0.9324, Spending_Score 54 65 37.5. The time taken to build the model (full training data) is 0 seconds. The output also shows the model and evaluation on training set, with clustered instances: 0 3 (60%), 1 2 (40%).

```
=== Run information ===
Scheme: weka.clusterers.SimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -W 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10
Relation: customer_data-weka.filters.unsupervised.attribute.Remove-RI-weka.filters.unsupervised.attribute.Normalize-S1.0-70.0
Instances: 5
Attributes: 2
Income
Spending_Score
Test mode: evaluate on training data

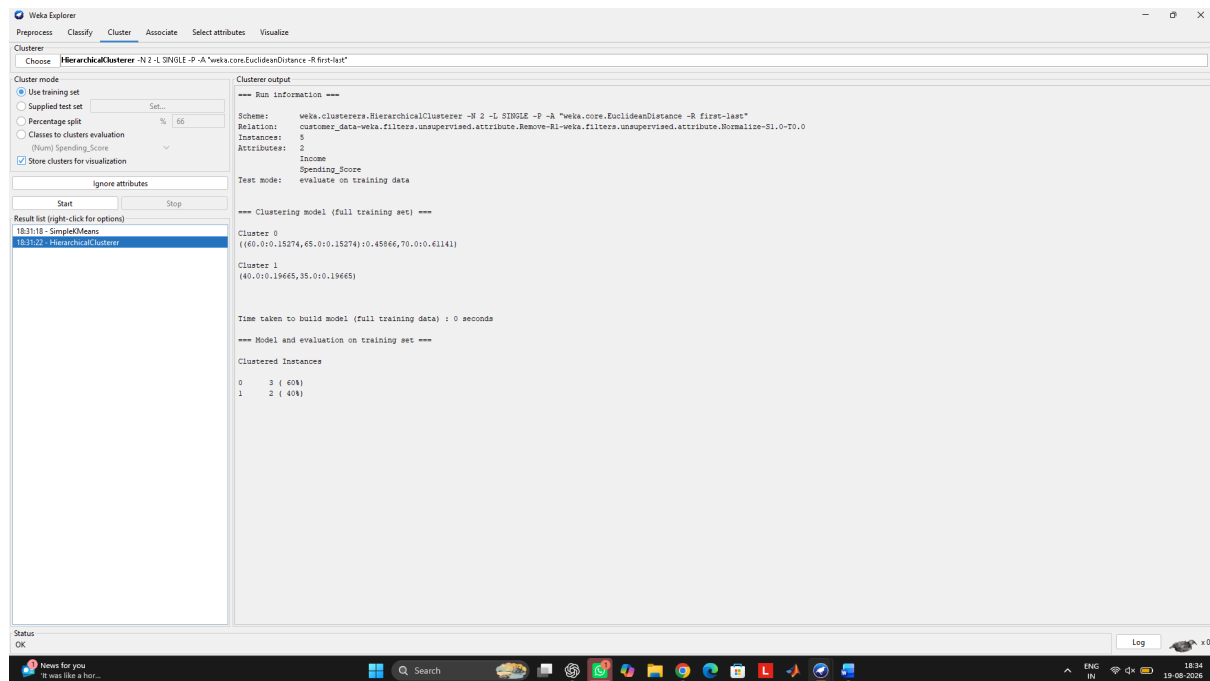
=== Clustering model (full training set) ===

KMeans
=====
Number of iterations: 2
Within cluster sum of squared errors: 0.27626737675347706
Initial starting points (random):
Cluster 0: 0.65
Cluster 1: 0.864865, 40
Missing values globally replaced with mean/mode
Final cluster centroids:
Attribute Full Data Cluster#
(5.0) (3.0) (2.0)
Income 0.5027 0.2162 0.9324
Spending_Score 54 65 37.5

Time taken to build model (full training data) : 0 seconds

=== Model and evaluation on training set ===

Clustered Instances
0 3 ( 60%)
1 2 ( 40%)
```

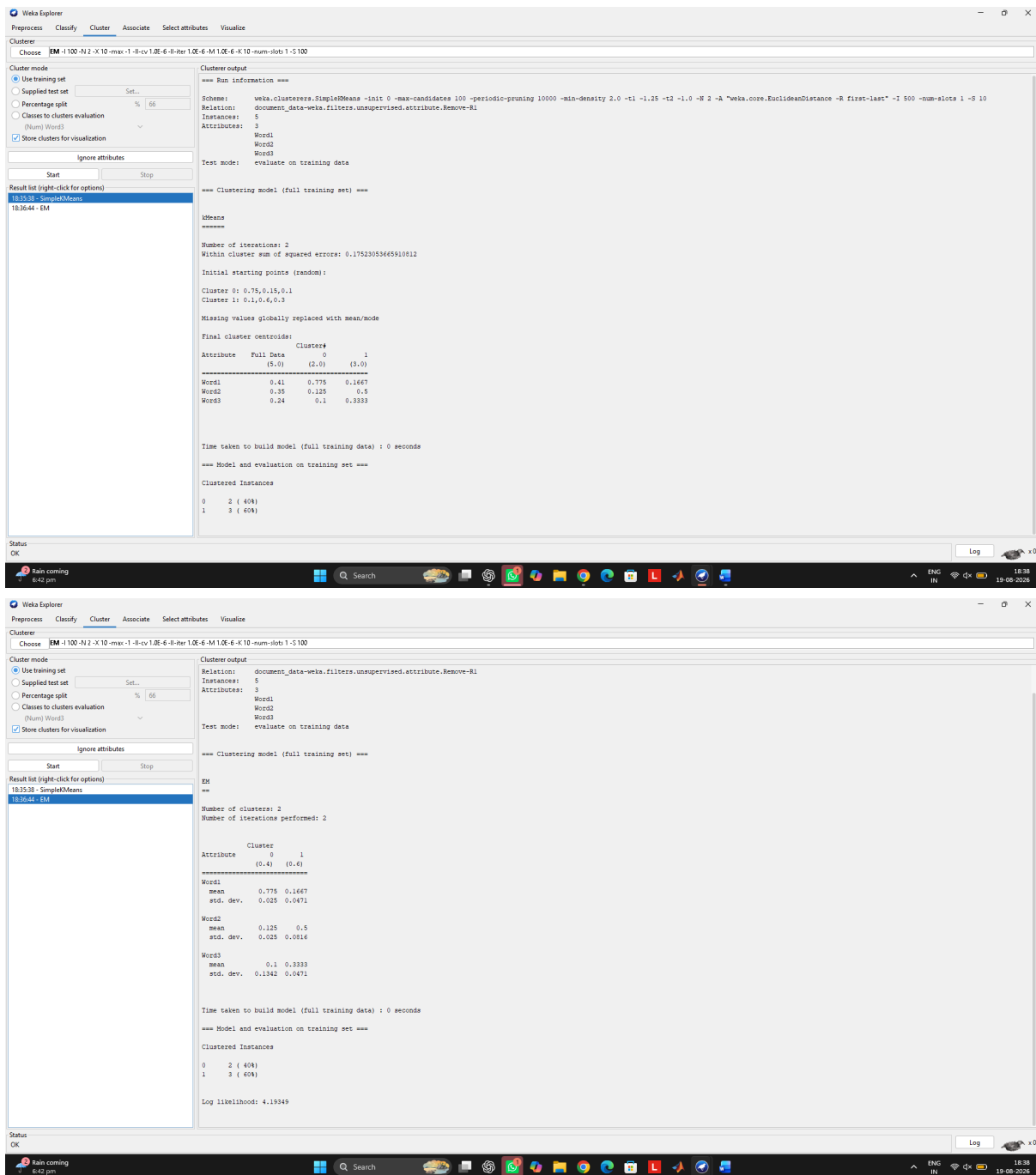


Q2

K-Means vs EM:

K-Means assigns each document to one cluster based on distance, so it works best when clusters are clearly separated. EM (Expectation-Maximization) models the probability distribution of each cluster and can assign probabilities to documents, making it more suitable when clusters overlap. For this document dataset, both methods identify two groups, but EM is preferable for overlapping document clusters because it provides probabilistic membership.

Method	Cluster handling
K-Means	Hard assignment; each document belongs to one cluster
EM	Probabilistic assignment; better for overlapping clusters



Q3

Euclidean distance measures straight-line distance between geographic points, while Manhattan distance measures the sum of absolute differences in latitude and longitude. Euclidean is generally suitable for geographic proximity, whereas Manhattan can be useful for grid-like movement. In this dataset, both metrics produced the same 2-cluster size distribution (3 points and 2 points), showing that the clustering is relatively stable for these points.

Metric	Result
Euclidean	2 clusters → 3 + 2 points

Metric	Result
Manhattan	2 clusters → 3 + 2 points

Weka Explorer
Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer: Choose **HierarchicalClusterer -N 2 -L SINGLE -P -A "weka.core.ManhattanDistance -R first-last"**

Cluster mode:
☒ Use training set
☐ Supplied test set
☐ Percentage split
☐ Classes to clusters evaluation
 (Name) Longitude
☒ Store clusters for visualization
 Ignore attributes
 Start Stop

Result list (right-click for options):
 184012 - SimpleKMeans
 184139 - HierarchicalClusterer
 184456 - HierarchicalClusterer

Clusterer output:
 --- Run information ---
 Scheme: weka.clusterers.SimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -W 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10
 Relation: geographic_data-weka.filters.unsupervised.attribute.Remove-S1-weka.filters.unsupervised.attribute.Normalize-S1.0-T0.0
 Instances: 5
 Attributes: 2
 Latitude
 Longitude
 Test mode: evaluate on training data
 --- Clustering model (full training set) ---
 KMeans
 Number of iterations: 4
 Within cluster sum of squared errors: 0.2482445759368771
 Initial starting points (random):
 Cluster 0: 0.418285, 80.3
 Cluster 1: 1, 80.25
 Missing values globally replaced with mean/mode
 Final cluster centroids:

Attribute	Full Data	Cluster#	0	1
	(5.0)	(2.0)	(3.0)	
Latitude	0.5265	0.1154	0.5205	
Longitude	80.248	80.21	80.2733	

 Time taken to build model (full training data) : 0 seconds
 --- Model and evaluation on training set ---
 Clustered Instances

0	2 (40%)
1	3 (60%)

Weka Explorer
Preprocess Classify **Cluster** Associate Select attributes Visualize

Clusterer: Choose **HierarchicalClusterer -N 2 -L SINGLE -P -A "weka.core.ManhattanDistance -R first-last"**

Cluster mode:
☒ Use training set
☐ Supplied test set
☐ Percentage split
☐ Classes to clusters evaluation
 (Name) Longitude
☒ Store clusters for visualization
 Ignore attributes
 Start Stop

Result list (right-click for options):
 184012 - SimpleKMeans
 184139 - HierarchicalClusterer
 184456 - HierarchicalClusterer

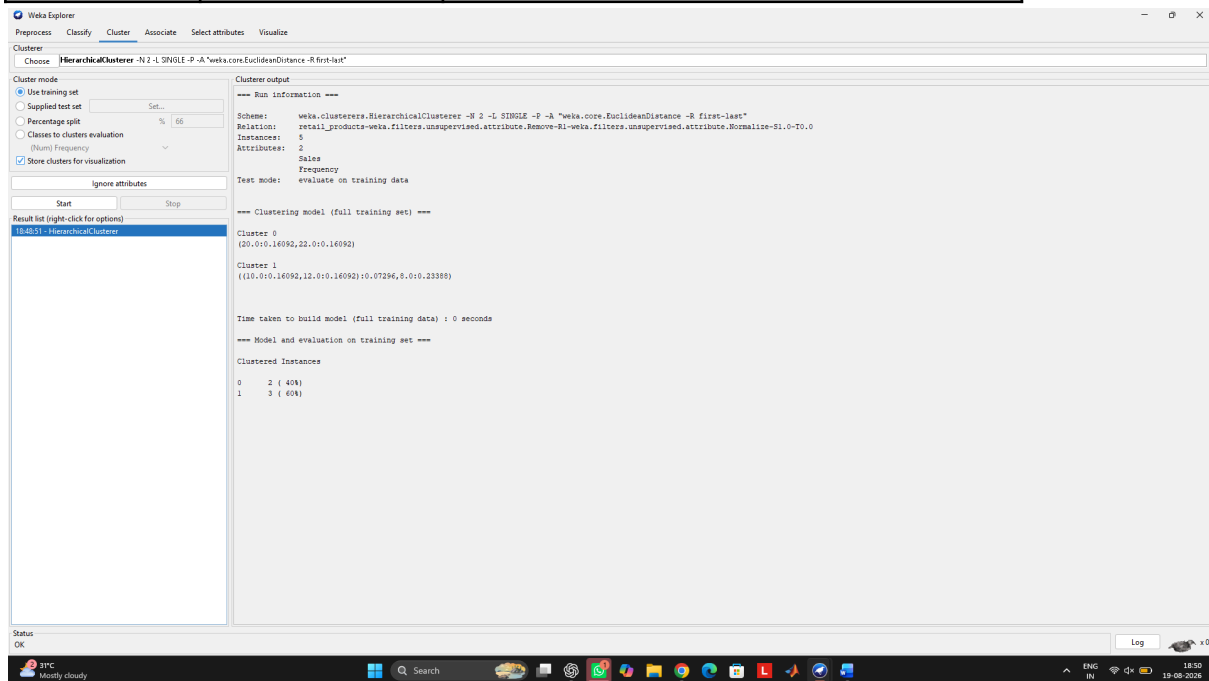
Clusterer output:
 --- Run information ---
 Scheme: weka.clusterers.HierarchicalClusterer -N 2 -L SINGLE -P -A "weka.core.ManhattanDistance -R first-last"
 Relation: geographic_data-weka.filters.unsupervised.attribute.Remove-S1-weka.filters.unsupervised.attribute.Normalize-S1.0-T0.0
 Instances: 5
 Attributes: 2
 Latitude
 Longitude
 Test mode: evaluate on training data
 --- Clustering model (full training set) ---
 Cluster 0
 ((80.2716, 35385, 80.25(0.35385); 0.17692, 80.3(0.53077))
 Cluster 1
 ((80.2210, 43077, 80.21(0.43077))
 Time taken to build model (full training data) : 0 seconds
 --- Model and evaluation on training set ---
 Clustered Instances

0	3 (60%)
1	2 (40%)

Q4

Agglomerative clustering starts with each product as an individual cluster and progressively merges similar products, while divisive clustering starts with all products in one cluster and recursively splits them. For this small retail dataset, both approaches can identify high-sales/high-frequency products such as P1 and P3 separately from lower-sales products. Agglomerative clustering is more suitable here because the dataset is small and its hierarchical structure is easy to interpret; divisive clustering can be useful when the main groups need to be identified through repeated splitting.

Approach	Process	Suitability
Agglomerative	Bottom-up merging	Suitable for this small dataset
Divisive	Top-down splitting	Suitable when major groups are split recursively



Q5

Centroid-based clustering, such as K-Means, groups data around cluster centroids and works well for compact numeric groups. Density-based clustering, such as MakeDensityBasedClusterer, models the probability distribution/density of the data and can provide more flexible cluster membership. For mixed data containing numeric attributes (Price, Rating) and a categorical attribute (Category), density-based methods can be useful, but appropriate handling of categorical data is required. In this dataset, both methods produced 2 clusters with a 2/3 split, showing similar grouping.

Method	WEKA result	Main advantage
K-Means	2 clusters: 2 + 3	Simple and fast for numeric data
MakeDensityBasedClusterer	2 clusters: 2 + 3	Uses probability/density information

Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize

Clusterer: Choose **MakeDensityBasedClusterer** -M 1.0E-6 -W weka.clusterers.SimpleKMeans --init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10

Cluster mode
☒ Use training set
☐ Supplied test set
☐ Percentage split
☐ Classes to clusters evaluation
 (Num) Rating
☒ Store clusters for visualization
 Ignore attributes
 Start Stop

Result list (right-click for options)
 18:53:55 - SimpleKMeans
 18:53:57 - MakeDensityBasedClusterer

Cluster output
 === Run information ===
 Scheme: weka.clusterers.SimpleKMeans -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10
 Relation: mixed_attributes-weka.filters.unsupervised.attribute.Remove-BI-weka.filters.unsupervised.attribute.Normalize-51.0-70.0
 Instances: 5
 Attributes: 3
 Price
 Category
 Rating
 Test mode: evaluate on training data
 === Clustering model (full training set) ===
 kMeans
 =====
 Number of iterations: 3
 Within cluster sum of squared errors: 0.0731018518518518
 Initial starting points (random):
 Cluster 0: 0.8,3.5
 Cluster 1: 0.166667,3.8
 Missing values globally replaced with mean/mode
 Final cluster centroids:

Attribute	Full Data	Cluster# 0	Cluster# 1
	(5.0)	(2.0)	(3.0)
Price	0.58	0.0833	0.9111
Category	A	B	A
Rating	4.22	3.65	4.6

 Time taken to build model (full training data) : 0 seconds
 === Model and evaluation on training set ===
 Clustered Instances

0	2 (40%)
1	3 (60%)

Status OK Log

Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize

Clusterer: Choose **MakeDensityBasedClusterer** -M 1.0E-6 -W weka.clusterers.SimpleKMeans --init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10

Cluster mode
☒ Use training set
☐ Supplied test set
☐ Percentage split
☐ Classes to clusters evaluation
 (Num) Rating
☒ Store clusters for visualization
 Ignore attributes
 Start Stop

Result list (right-click for options)
 18:53:55 - SimpleKMeans
 18:53:57 - MakeDensityBasedClusterer

Cluster output
 Cluster 0: 0.8,3.5
 Cluster 1: 0.166667,3.8
 Missing values globally replaced with mean/mode
 Final cluster centroids:

Attribute	Full Data	Cluster# 0	Cluster# 1
	(5.0)	(2.0)	(3.0)
Price	0.58	0.0833	0.9111
Category	A	B	A
Rating	4.22	3.65	4.6

 Fitted estimators (with MC estimates of variance):
 Cluster: 0 Prior probability: 0.4286
 Attribute: Price
 Normal Distribution. Mean = 0.0833 StdDev = 0.0833
 Attribute: Category
 Discrete Estimator. Counts = 1 3 (Total = 4)
 Attribute: Rating
 Normal Distribution. Mean = 3.65 StdDev = 0.15
 Cluster: 1 Prior probability: 0.5714
 Attribute: Price
 Normal Distribution. Mean = 0.9111 StdDev = 0.0455
 Attribute: Category
 Discrete Estimator. Counts = 4 1 (Total = 5)
 Attribute: Rating
 Normal Distribution. Mean = 4.6 StdDev = 0.0816
 Time taken to build model (full training data) : 0 seconds
 === Model and evaluation on training set ===
 Clustered Instances

0	2 (40%)
1	3 (60%)

 Log likelihood: 1.10309

Status OK Log